

Plan de Pruebas del Sistema — Cámaras-IA

Proyecto de Título · Duoc UC Versión: 1.0 · **Fecha:** Junio 2026 **Equipo:** Ignacio Fuenzalida Ramírez · Nicolas Alejandro Arcos Alfaro

1. Introducción y Objetivos

Este documento describe el **Plan de Pruebas** del sistema Cámaras-IA, un ecosistema de videovigilancia inteligente basado en arquitectura híbrida Cloud-Edge. Su propósito es asegurar la calidad del software entregado mediante la ejecución sistemática de pruebas funcionales, de seguridad, de integración y de rendimiento, siguiendo el **estándar ISO 25010**.

1.1 Objetivos del Plan de Pruebas

#	Objetivo
1	Verificar que todos los endpoints de la API REST respondan correctamente con los datos y códigos HTTP esperados
2	Validar que el sistema de autenticación (JWT + bcrypt) proteja correctamente los recursos según el rol del usuario
3	Comprobar que el pipeline de detección IA (YOLO + LLaVA) genere alertas estructuradas con campos completos
4	Verificar que las notificaciones de Telegram lleguen al canal destino configurado
5	Asegurar que el sistema de descubrimiento de IPs de cámaras por ARP funcione correctamente
6	Validar la resiliencia de los heartbeats y el monitoreo de salud de servicios

1.2 Alcance

Las pruebas cubren los siguientes módulos del sistema:

- **Módulo de Autenticación:** `auth.py` — bcrypt, JWT, roles (admin / operador / visor)
- **Módulo de API REST:** `api.py` — 35+ endpoints FastAPI
- **Módulo de Detección IA:** `detector.py`, `analizador.py`, `analizador2.py`
- **Módulo de Notificaciones:** `telegram/` — Worker de alertas Telegram
- **Módulo de Infraestructura:** `salud.py`, `base_datos/db.py` — PostgreSQL, Redis, MinIO

2. Plan de Pruebas

2.1 Módulo de Autenticación y Control de Acceso

CP-AUTH-01 · Login con credenciales válidas

Campo	Detalle
Módulo	Autenticación
Precondición	Usuario admin activo en la base de datos PostgreSQL
Entrada	POST /auth/login con { "email": "admin@camaras.ia", "password": "Admin2024!" }
Resultado Esperado	HTTP 200 · Cuerpo contiene <code>access_token</code> (JWT válido) y <code>token_type</code> : "bearer"
Criterio de Éxito	Token puede decodificarse con la clave secreta del sistema

CP-AUTH-02 · Login con contraseña incorrecta

Campo	Detalle
Módulo	Autenticación
Entrada	POST /auth/login con contraseña errónea
Resultado Esperado	HTTP 401 · Mensaje " <code>Credenciales inválidas</code> "
Criterio de Éxito	No se devuelve token bajo ninguna circunstancia

CP-AUTH-03 · Acceso a endpoint protegido sin token

Campo	Detalle
Módulo	Autenticación
Entrada	GET /usuarios sin cabecera <code>Authorization</code>
Resultado Esperado	HTTP 401 · " <code>Not authenticated</code> "
Criterio de Éxito	El sistema rechaza la solicitud antes de ejecutar lógica de negocio

CP-AUTH-04 · Control de acceso por rol (Operador intenta acción de Admin)

Campo	Detalle
Módulo	Autenticación · Control de Acceso
Precondición	Usuario con rol <code>operador</code> autenticado
Entrada	GET /usuarios con token de operador (listado de usuarios requiere admin)
Resultado Esperado	HTTP 403 · " <code>Requiere rol admin</code> "
Criterio de Éxito	El sistema distingue correctamente los roles y bloquea la acción

CP-AUTH-05 · Operador puede cambiar credenciales RTSP

Campo	Detalle
Módulo	Autenticación · Control de Acceso
Precondición	Usuario con rol operador autenticado
Entrada	PATCH <code>/camaras/{id}/credenciales</code> con token de operador
Resultado Esperado	HTTP 200 · Credenciales actualizadas en base de datos
Criterio de Éxito	El rol operador tiene permiso explícito para esta acción

CP-AUTH-06 · Expiración de token JWT

Campo	Detalle
Módulo	Autenticación
Entrada	Token JWT con fecha de expiración pasada
Resultado Esperado	HTTP 401 · " Token expirado "
Criterio de Éxito	El sistema no acepta tokens vencidos

2.2 Módulo de API REST — Cámaras y Configuración

CP-API-01 · Listado de cámaras activas

Campo	Detalle
Módulo	API · Cámaras
Entrada	GET <code>/camaras</code> con token válido
Resultado Esperado	HTTP 200 · Array de objetos con campos <code>id</code> , <code>nombre</code> , <code>ip_actual</code> , <code>estado_salud</code> , <code>hls_url</code>
Criterio de Éxito	La URL HLS dinámica se construye desde <code>configuracion_sistema.MEDIAMTX_HLS_DYNAMIC</code>

CP-API-02 · Obtener cámara por ID

Campo	Detalle
Módulo	API · Cámaras
Entrada	GET <code>/camaras/{camara_id}</code> con ID válido
Resultado Esperado	HTTP 200 · Objeto cámara con todos los campos
Criterio de Éxito	Los campos <code>usuario_rtsp</code> y <code>password_rtsp</code> NO se exponen en texto plano

CP-API-03 · Endpoint de salud del sistema

Campo	Detalle
Módulo	API · Meta
Entrada	GET /health sin autenticación
Resultado Esperado	HTTP 200 · Objeto con estado de cada servicio: postgres, redis, minio, api
Criterio de Éxito	Responde en menos de 500ms

CP-API-04 · Listado de alertas con filtros

Campo	Detalle
Módulo	API · Alertas
Entrada	GET /alertas?nivel=alto&limit=10 con token válido
Resultado Esperado	HTTP 200 · Máximo 10 alertas filtradas por nivel "alto"
Criterio de Éxito	Paginación funciona correctamente con parámetros limit y offset

CP-API-05 · Stream SSE de alertas en tiempo real

Campo	Detalle
Módulo	API · Realtime
Entrada	GET /alertas/stream con token válido (Server-Sent Events)
Resultado Esperado	Conexión persistente · Eventos llegan en formato data: {...}\n\n
Criterio de Éxito	El frontend recibe nuevas alertas sin necesidad de polling

CP-API-06 · Eliminación de alerta (solo Admin)

Campo	Detalle
Módulo	API · Alertas
Entrada	DELETE /alertas/{alerta_id} con token admin
Resultado Esperado	HTTP 200 · Alerta eliminada de la base de datos
Criterio de Éxito	Un operador recibe HTTP 403 al intentar la misma acción

CP-API-07 · Métricas del sistema

Campo	Detalle
Módulo	API · Meta
Entrada	GET /sistema/metricas con token válido
Resultado Esperado	HTTP 200 · Objeto con alertas_totales, grabaciones_totales, camaras_online
Criterio de Éxito	Los contadores coinciden con los datos reales en la base de datos

2.3 Módulo de Gestión de Usuarios (CRUD)

CP-USR-01 · Crear usuario nuevo (Admin)

Campo	Detalle
Módulo	API · Usuarios
Entrada	POST /usuarios con { "email": "op@test.com", "password": "Test123!", "rol": "operador" }
Resultado Esperado	HTTP 201 · Usuario creado con password hashado en bcrypt (nunca en texto plano)
Criterio de Éxito	El campo password_hash en la DB comienza con \$2b\$ (identificador bcrypt)

CP-USR-02 · Editar rol de usuario propio (protección)

Campo	Detalle
Módulo	API · Usuarios
Precondición	Usuario admin autenticado
Entrada	PUT /usuarios/{propio_id} intentando cambiar su propio rol
Resultado Esperado	HTTP 403 · "No puedes cambiar tu propio rol"
Criterio de Éxito	El sistema previene la auto-degradación accidental de administradores

CP-USR-03 · Eliminar usuario propio (protección)

Campo	Detalle
Módulo	API · Usuarios
Entrada	DELETE /usuarios/{propio_id} con token propio
Resultado Esperado	HTTP 403 · Prohibición de auto-eliminación
Criterio de Éxito	Siempre debe existir al menos un administrador activo

2.4 Módulo de Detección IA (Pipeline YOLO + LLaVA)

[!NOTE] Estas pruebas se ejecutan con el detector corriendo localmente en Windows con acceso físico a las cámaras.

CP-IA-01 · Detección de persona en horario diurno

Campo	Detalle
Módulo	Detección IA · YOLO
Precondición	Cámara Sonoff online · Detector v2 (<code>start-seguridad.bat</code>) corriendo
Entrada	Persona entrando al área de cobertura de la cámara
Resultado Esperado	Bounding box dibujado en menos de 2 segundos · Alerta generada en la tabla <code>alertas</code>
Criterio de Éxito	Latencia punta a punta (detección → alerta en frontend) < 2.0 segundos

CP-IA-02 · Análisis semántico LLaVA sobre frame detectado

Campo	Detalle
Módulo	Detección IA · LLaVA
Precondición	Ollama corriendo localmente con modelo LLaVA cargado
Entrada	Frame con persona capturado tras CP-IA-01
Resultado Esperado	Alerta actualizada con campos: <code>descripcion</code> (texto), <code>sospechoso</code> (bool), <code>nivel</code> (bajo/medio/alto), <code>personas</code> (int), <code>acciones</code> (str)
Criterio de Éxito	Todos los campos están presentes en el JSON resultante · Tiempo de análisis registrado en <code>tiempo_analisis</code>

CP-IA-03 · Clasificación de nivel de alerta

Campo	Detalle
Módulo	Detección IA
Entrada	Persona con rostro cubierto frente a la cámara
Resultado Esperado	<code>nivel = "alto"</code> · <code>sospechoso = true</code>
Criterio de Éxito	La IA clasifica correctamente el comportamiento sospechoso

CP-IA-04 · Descubrimiento de IP de cámara por ARP

Campo	Detalle
Módulo	Infraestructura · Red
Precondición	Cámara conectada a la red local
Entrada	Ejecución de <code>start-seguridad.bat</code> con la cámara encendida
Resultado Esperado	IP correcta descubierta por MAC address en tabla ARP · JSON escrito en <code>camaras_ips_descubiertas.json</code>
Criterio de Éxito	Sistema encuentra la cámara sin intervención manual

CP-IA-05 · Fallback a IP de respaldo cuando cámara no está en ARP

Campo	Detalle
Módulo	Infraestructura · Red
Precondición	Cámara principal offline o no visible en ARP
Entrada	Inicio del detector
Resultado Esperado	Log <code>[WARN] Camara_Principal: no encontrada en ARP. Usando IP de respaldo 192.168.1.15</code>
Criterio de Éxito	El sistema no colapsa; usa la IP de respaldo de la base de datos

2.5 Módulo de Notificaciones (Telegram Worker)

CP-TEL-01 · Conexión del bot al iniciar el worker

Campo	Detalle
Módulo	Telegram Worker
Precondición	<code>TELEGRAM_BOT_TOKEN</code> configurado en tabla <code>configuracion_sistema</code>
Entrada	Inicio del contenedor <code>camaras-telegram</code> en Oracle
Resultado Esperado	Log:  Bot conectado: <code>@camaraAi_bot</code> (Camaras-Seguridad-alertas)
Criterio de Éxito	El contenedor no colapsa en bucle de reintentos

CP-TEL-02 · Envío de notificación ante alerta de nivel medio/alto

Campo	Detalle
Módulo	Telegram Worker
Precondición	Bot conectado · <code>TELEGRAM_CHAT_ID</code> configurado correctamente
Entrada	Alerta generada con <code>nivel = "alto"</code>

Campo	Detalle
Resultado Esperado	Mensaje recibido en Telegram en menos de 10 segundos post-generación de alerta
Criterio de Éxito	El mensaje contiene: nombre de cámara, nivel, descripción y timestamp

CP-TEL-03 · Worker sin TELEGRAM_BOT_TOKEN en base de datos

Campo	Detalle
Módulo	Telegram Worker
Entrada	Inicio del contenedor sin token configurado
Resultado Esperado	Log: ✖ Falta TELEGRAM_BOT_TOKEN en configuracion_sistema · Worker reinicia con backoff exponencial
Criterio de Éxito	El sistema falla de forma controlada y no genera errores críticos en otros servicios

2.6 Módulo de Infraestructura y Resiliencia

CP-INF-01 · Heartbeats de servicios hacia la base de datos

Campo	Detalle
Módulo	Salud · Infraestructura
Entrada	Sistema corriendo por más de 60 segundos
Resultado Esperado	Tabla salud_servicios con registros actualizados para: detector, yolo, llava, telegram_worker, api
Criterio de Éxito	Campo segundos_sin_reporte menor a 60 para cada servicio activo

CP-INF-02 · Auto-descubrimiento de URL del túnel (Cloudflare)

Campo	Detalle
Módulo	Infraestructura · Tunneling
Entrada	Ejecución de start-mediatrix.bat
Resultado Esperado	URL pública de Cloudflare detectada y guardada en configuracion_sistema.MEDIATRIX_HLS_DYNAMIC
Criterio de Éxito	El frontend obtiene la URL HLS correcta sin reiniciar el backend

CP-INF-03 · Caída y recuperación del servicio Redis

Campo	Detalle
Módulo	Infraestructura · Redis
Entrada	Redis detenido mientras el detector está corriendo
Resultado Esperado	El sistema continúa operando (modo degradado) · Heartbeat reporta estado = "degradado" para redis
Criterio de Éxito	La detección de personas no se interrumpe por la caída de Redis

3. Resultados de Ejecución de Pruebas

Las pruebas se ejecutaron entre el **Sprint 7 y el Sprint 9** del proyecto, en el ambiente de producción Oracle Cloud (backend) y en la PC local de desarrollo (detector).

3.1 Resumen Ejecutivo

Módulo	Casos Planificados	Aprobados 	Fallidos 	Pendientes 
Autenticación	6	5	1	0
API REST	7	7	0	0
Gestión de Usuarios	3	3	0	0
Detección IA	5	4	1	0
Notificaciones Telegram	3	2	1	0
Infraestructura	3	3	0	0
TOTAL	27	24	3	0

[!NOTE] **Tasa de aprobación: 88,9%** — Los 3 casos fallidos generaron mejoras implementadas que se describen en la Sección 4.

3.2 Resultados Detallados

Módulo de Autenticación

ID	Resultado	Observaciones
CP-AUTH-01	 Aprobado	Token JWT generado correctamente con sub, email y rol
CP-AUTH-02	 Aprobado	HTTP 401 recibido correctamente
CP-AUTH-03	 Aprobado	HTTP 401 sin token

ID	Resultado	Observaciones
CP-AUTH-04	 Fallido	El operador podía modificar credenciales RTSP, que estaban protegidas con <code>require_admin</code> . El UML de casos de uso indicaba que esta acción corresponde al operador
CP-AUTH-05	 Aprobado	<i>(Posterior a la corrección implementada)</i>
CP-AUTH-06	 Aprobado	Tokens expirados rechazados correctamente

Módulo de API REST

ID	Resultado	Observaciones
CP-API-01	 Aprobado	URL HLS dinámica inyectada correctamente desde DB
CP-API-02	 Aprobado	Campos RTSP no expuestos en texto plano
CP-API-03	 Aprobado	Respuesta en < 200ms
CP-API-04	 Aprobado	Filtros y paginación funcionan
CP-API-05	 Aprobado	SSE entrega eventos en tiempo real
CP-API-06	 Aprobado	Control de roles correcto
CP-API-07	 Aprobado	Métricas correctas

Módulo de Gestión de Usuarios

ID	Resultado	Observaciones
CP-USR-01	 Aprobado	Password hasheado con bcrypt (<code>\$2b\$12\$...</code>)
CP-USR-02	 Aprobado	Protección de auto-edición de rol activa
CP-USR-03	 Aprobado	Protección de auto-eliminación activa

Módulo de Detección IA

ID	Resultado	Observaciones
CP-IA-01	 Aprobado	Latencia promedio medida: 1.3 segundos
CP-IA-02	 Aprobado	Todos los campos presentes. Tiempo de análisis LLaVA en CPU: ~120 segundos
CP-IA-03	 Aprobado	LLaVA clasificó correctamente conductas sospechosas en 4 de 5 pruebas

ID	Resultado	Observaciones
CP-IA-04	 Aprobado	IP descubierta por ARP en todas las pruebas
CP-IA-05	 Fallido	Cuando la cámara no está en ARP, el flujo RTSP usa la IP de respaldo pero el detector no generaba alertas de nivel "bajo" hacia Telegram, ya que el filtro solo permitía "alto" y "medio"

Módulo de Notificaciones Telegram

ID	Resultado	Observaciones
CP-TEL-01	 Aprobado	Bot se conectó exitosamente tras configurar el token
CP-TEL-02	 Fallido	Las notificaciones no llegaban. Se identificaron dos causas: (1) <code>TELEGRAM_CHAT_ID</code> tenía valor <code>0</code> en la base de datos y (2) el filtro de niveles excluía alertas de bajo nivel
CP-TEL-03	 Aprobado	Worker reinicia con backoff exponencial sin afectar otros servicios

Módulo de Infraestructura

ID	Resultado	Observaciones
CP-INF-01	 Aprobado	Heartbeats activos para todos los servicios
CP-INF-02	 Aprobado	URL de Cloudflare actualizada automáticamente en cada arranque
CP-INF-03	 Aprobado	Sistema operó en modo degradado correctamente

4. Mejoras Implementadas a Partir de los Resultados

4.1 Mejora #1: Control de Acceso por Rol para Credenciales RTSP

Caso fallido: CP-AUTH-04

Problema identificado: El endpoint `PATCH /camaras/{id}/credenciales` y `POST /sistema/reload-camaras` tenían el decorador `require_admin`, pero el diseño UML del sistema (Diagrama de Casos de Uso) establecía que la acción **"Cambiar Credenciales RTSP"** corresponde al rol **Operador**, no al Administrador.

Solución implementada:

1. Se creó una nueva dependencia `require_operator` en `auth.py` que acepta roles `admin` y `operador`.
2. Se actualizaron los endpoints para usar `require_operator` en lugar de `require_admin`.
3. Se verificó que usuarios con rol `visor` continúan siendo bloqueados (HTTP 403).

```
# auth.py – Nueva función agregada
def require_operator(user: dict = Depends(get_current_user)) -> dict:
    """Exige rol operador o admin."""
    if user.get("rol") not in ("admin", "operador"):
        raise HTTPException(403, "Requiere rol operador o admin")
    return user
```

Resultado: CP-AUTH-05 aprobado. Operadores pueden cambiar credenciales RTSP sin necesitar privilegios de administrador.

4.2 Mejora #2: Configuración Correcta del Chat ID de Telegram

Caso fallido: CP-TEL-02 (causa 1)

Problema identificado: Al revisar la base de datos en Oracle, se encontró que `TELEGRAM_CHAT_ID` tenía el valor `0` (cero), que es un identificador inválido en la API de Telegram. El bot intentaba enviar mensajes al chat `0`, que no existe, por lo que la API de Telegram rechazaba silenciosamente las notificaciones.

Diagnóstico:

```
SELECT clave, valor
FROM configuracion_sistema
WHERE clave IN ('TELEGRAM_BOT_TOKEN', 'TELEGRAM_CHAT_ID');
-- Resultado: TELEGRAM_CHAT_ID | 0 ← INVÁLIDO
```

Solución implementada: Se actualizó el `TELEGRAM_CHAT_ID` con el identificador real del usuario destino usando el bot `@userinfobot` de Telegram para obtener el ID numérico:

```
UPDATE configuracion_sistema
SET valor = '7416452486'
WHERE clave = 'TELEGRAM_CHAT_ID';
```

Resultado: Las notificaciones llegaron al celular destino correctamente.

4.3 Mejora #3: Inclusión del Nivel "Bajo" en el Filtro de Notificaciones

Caso fallido: CP-IA-05 y CP-TEL-02 (causa 2)

Problema identificado: En `detector.py`, la lógica que decide si una alerta se envía a Telegram filtraba únicamente alertas de nivel `"alto"` y `"medio"`. Las alertas de nivel `"bajo"` (persona detectada sin comportamiento sospechoso) nunca llegaban a la cola de notificaciones, lo que hacía imposible probar el sistema de extremo a extremo en condiciones normales.

Solución implementada: Se modificó el filtro en `detector.py` para incluir el nivel `"bajo"`:

```
# detector.py – Antes
if nivel in ("alto", "medio") or sospechoso:
    encolar_notificacion(...)

# detector.py – Después
if nivel in ("alto", "medio", "bajo") or sospechoso:
    encolar_notificacion(...)
```

Resultado: Las notificaciones de Telegram funcionan para cualquier nivel de alerta, permitiendo verificar el pipeline completo de extremo a extremo.

5. Conclusiones

Las pruebas ejecutadas permitieron identificar y corregir 3 defectos funcionales del sistema antes de la entrega final:

#	Defecto	Severidad	Estado
1	Control de acceso incorrecto para credenciales RTSP	Alta	✓ Corregido
2	Chat ID de Telegram inválido (valor 0)	Alta	✓ Corregido
3	Filtro de niveles excluía alertas de nivel "bajo"	Media	✓ Corregido

5.1 Lecciones Aprendidas

- **La seguridad por roles debe mapearse explícitamente desde el diseño UML:** Una inconsistencia entre el diagrama de casos de uso y el código de autorización generó el defecto CP-AUTH-04. En proyectos futuros, se recomienda un checklist de permisos al final de cada Sprint.
- **Los datos de configuración en base de datos deben tener valores válidos por defecto:** El valor 0 para el Chat ID debería haberse detectado con una restricción CHECK en la base de datos.
- **Las pruebas de extremo a extremo son fundamentales:** Solo ejecutando el pipeline completo (detección → alerta → Telegram) fue posible identificar los tres defectos simultáneamente.

5.2 Trabajo Futuro

Área	Propuesta
Pruebas automatizadas	Implementar suite con pytest para los endpoints de la API REST
Pruebas de carga	Simular 10+ conexiones concurrentes al endpoint SSE /alertas/stream
Pruebas nocturnas	Validar que la tasa de falsos negativos sea < 5% en condiciones de baja luminosidad
Pruebas de failover	Simular caída del nodo Oracle y verificar recuperación automática